

Model-Driven Web Application Code Generation: A Comparative Study of Intelligent Model-Driven Methods and Pure Large Language Model Approaches

Course: Software Requirements Analysis and Design
Student Project Report
May 24, 2026

Abstract

The rapid advancement of Large Language Models (LLMs) has opened new possibilities for automated code generation in software engineering. However, the effectiveness of directly using LLMs versus combining them with formal software engineering methods remains an open question. This report presents a comparative study between an **Intelligent Model-Driven Method**—where domain models define the application architecture and LLMs fill in business logic within pre-generated code skeletons—and a **Pure LLM Method**—where natural language descriptions are fed directly to LLMs for full-stack code generation. We implement both approaches for a Student Course Selection Management System, generating complete web applications consisting of FastAPI backends and React frontends across three different system scales (small, medium, and large). Our experimental results demonstrate that the model-driven approach achieves a **100% compilability rate** with **98.4% cross-run consistency**, while the pure LLM approach achieves only **33.3% compilability** with **48.1% consistency**. Additionally, we propose a novel **Bidirectional Feedback Mechanism** that automatically detects and corrects code errors by feeding compilation and import errors back to the LLM within model-defined constraints. This innovation enables the model-driven method to converge to error-free code within a single feedback iteration on average. Our findings suggest that combining formal modeling techniques with LLM capabilities provides significant benefits in terms of code quality, consistency, and reliability compared to unconstrained LLM generation.

Contents

1	Introduction	6
1.1	Background and Motivation	6
1.2	Research Questions	6
1.3	Contributions	6
1.4	Report Structure	7
2	Background and Related Work	8
2.1	Model-Driven Engineering	8
2.1.1	Model-Driven Architecture (MDA)	8
2.1.2	Model-to-Text Transformations	8
2.2	Large Language Models for Code Generation	8
2.2.1	Evolution of Code LLMs	8
2.2.2	Limitations of Pure LLM Generation	8
2.3	Hybrid Approaches	9
2.4	Feedback-Driven Code Improvement	9
3	Methodology	10
3.1	Overview	10
3.2	Method A: Intelligent Model-Driven Approach	10
3.2.1	Stage 1: Domain Model Definition	10
3.2.2	Stage 2: Model Parsing and Validation	10
3.2.3	Stage 3: Deterministic Code Skeleton Generation	11
3.2.4	Stage 4: LLM Business Logic Completion	11
3.2.5	Stage 5: Bidirectional Feedback Loop (Innovation)	11
3.3	Method B: Pure LLM Baseline	12
3.4	Evaluation Framework	12
3.4.1	Metrics	12
3.4.2	Experiment Design	12
4	System Design and Implementation	13
4.1	Architecture Overview	13
4.2	Domain Model Definition	13
4.2.1	YAML Schema Design	13
4.2.2	Model Sizes	14
4.3	Intermediate Representation (IR)	15
4.4	Code Skeleton Generation	16
4.4.1	Jinja2 Template Engine	16
4.4.2	SQL Generator	16
4.4.3	FastAPI Generator	16
4.4.4	React Generator	16
4.5	LLM Integration	17
4.5.1	DeepSeek API Client	17
4.5.2	Prompt Engineering for Filler Mode	17
4.5.3	Baseline Prompt Design	17
4.6	Bidirectional Feedback Mechanism	17
4.7	Evaluation Components	18
4.7.1	Structure Checker	18
4.7.2	Compilability Checker	18
4.7.3	Consistency Checker	19

5	Case Study: Student Course Selection System	20
5.1	Domain Description	20
5.2	Small Model Details	20
5.3	Medium Model Details	20
5.4	Large Model Details	21
6	Experimental Setup	22
6.1	Hardware and Software Environment	22
6.2	Experiment Execution	22
6.3	Data Collection	22
7	Experimental Results and Evaluation	23
7.1	Structure Correctness	23
7.2	Compilability	23
7.2.1	Error Analysis for Pure LLM	23
7.3	Consistency	24
7.4	Feedback Loop Performance	24
7.5	Generation Time	24
7.6	Scalability Analysis	25
8	Discussion	26
8.1	Interpretation of Results	26
8.1.1	Structural Fidelity	26
8.1.2	Compilability: The Critical Gap	26
8.1.3	Consistency and Reproducibility	26
8.1.4	Feedback Loop Effectiveness	26
8.2	Answering Research Questions	26
8.3	Innovation Contribution	27
8.4	Limitations	27
8.5	Future Work	27
9	Development Practices and Code Assistant Usage	29
9.1	Development Workflow	29
9.2	Code Assistant Usage	29
9.2.1	Usage Modes	29
9.2.2	Interaction Patterns	29
9.2.3	Skills and Plugins	30
9.3	CI/CD Integration	30
9.4	Test Suite	30
9.5	Reproducibility	30
10	Model Definitions	32
10.1	Small Model (models/small.yaml)	32
10.2	Medium Model (models/medium.yaml)	33
10.3	Large Model (models/large.yaml)	33
11	Implementation Challenges and Solutions	34
11.1	SQL vs. SQLAlchemy Type Mismatch	34
11.2	Relative Import Failures	34
11.3	String Default Value Quoting in Schemas	34
11.4	CURRENT_TIMESTAMP as Python Default	34
11.5	Feedback Loop Over-Modification	35

11.6	Template Whitespace Management	35
11.7	Node_modules Scanning in Consistency Checker	35
11.8	Multi-line Description in Generated Code	35
11.9	Frontend TypeScript Type Errors	35
12	Prompt Engineering Analysis	37
12.1	Filler Prompt Design Principles	37
12.1.1	Positive Instructions (Must-Do)	37
12.1.2	Negative Instructions (Must-Not-Do)	37
12.1.3	Context Provision	37
12.2	Baseline Prompt Design	37
12.3	Feedback Fix Prompt Design	38
12.4	Lessons Learned	38
13	Extended Related Work	39
13.1	Model-Driven Engineering: Historical Context	39
13.1.1	Domain-Specific Modeling (DSM)	39
13.1.2	Executable UML	39
13.2	Template-Based Code Generation	39
13.2.1	Template Engines	39
13.2.2	Scaffolding Frameworks	39
13.3	LLM-Assisted Software Engineering	39
13.3.1	Code Completion and Generation	39
13.3.2	Program Repair	40
13.3.3	Self-Debugging and Self-Reflection	40
13.4	Formal Methods and LLMs	40
14	Detailed Template Design	41
14.1	SQL Template (schema.sql.j2)	41
14.2	FastAPI Models Template (models.py.j2)	41
14.3	FastAPI Routes Template (routes.py.j2)	41
14.4	Pydantic Schemas (Generated in Python)	42
14.5	React Templates Overview	42
14.6	TypeScript Type Generation	42
15	Generated Code Examples	43
15.1	Generated SQL Schema	43
15.2	Generated SQLAlchemy Models	43
15.3	Generated API Routes (After LLM Filling)	44
15.4	Generated React Component	44
16	Complete Experiment Data	46
16.1	Experiment Run Summary	46
16.2	Model-Driven Structure Details	46
16.3	Pure LLM Error Analysis	46
16.4	Consistency Analysis	46
17	Project Repository Information	47
17.1	Repository Structure	47
17.2	Reproducibility Checklist	47
17.3	Test Execution Instructions	47
17.4	CI/CD Pipeline	47

18 Detailed Error Case Study	49
18.1 Case 1: SQLAlchemy Type Name Errors	49
18.2 Case 2: Pydantic Schema Default Values	49
18.3 Case 3: Import Structure Fragility	49
18.4 Case 4: Feedback Loop Destructive Behavior	50
18.5 Case 5: Combinatorial Explosion in Consistency Checking	50
19 Scalability and Performance Analysis	51
19.1 Generation Time Composition	51
19.2 Model Size Scaling	51
19.3 API Reliability Considerations	51
20 Use of Code Assistant (Claude Code)	52
20.1 Assistant Configuration	52
20.2 Workflow Patterns	52
20.3 Key Contributions of the Assistant	52
20.4 Human Oversight	53
21 Complete Model Definitions	54
21.1 Medium Model (models/medium.yaml)	54
21.2 Large Model (models/large.yaml)	57
22 Conclusion	63
22.1 Summary of Findings	63
22.2 Practical Implications	63
22.3 Final Remarks	63

1 Introduction

1.1 Background and Motivation

Software development has entered a transformative era with the emergence of Large Language Models (LLMs) such as GPT-4, Claude, and DeepSeek. These models have demonstrated remarkable capabilities in generating source code from natural language descriptions, leading to a surge of interest in AI-assisted software engineering tools. However, the practical application of LLMs in real-world software development faces significant challenges.

When developers use LLMs to generate complete applications from scratch, several issues commonly arise: (1) **structural inconsistency**—the generated code may not follow consistent architectural patterns across multiple invocations; (2) **import and dependency errors**—LLMs frequently produce code with incorrect module imports or missing dependencies; (3) **business rule violations**—complex domain constraints may be overlooked or incorrectly implemented; and (4) **low reproducibility**—the same prompt may yield substantially different code structures across multiple runs.

Model-Driven Engineering (MDE) has long been established as a systematic approach to software development, where abstract models serve as the primary artifacts from which concrete implementations are derived. MDE techniques offer well-defined structures, deterministic transformations, and verifiable consistency between models and generated code.

This project investigates whether combining MDE principles with LLM capabilities can address the limitations of pure LLM-based code generation. We propose an **Intelligent Model-Driven Method** where domain models defined in YAML specify the application’s data model, relationships, and business rules. A template-based code generator produces deterministic code skeletons (database schemas, API routes, and user interface components), while an LLM fills in the business logic at designated markers. Crucially, the LLM operates within the constraints defined by the model, ensuring that the overall architecture remains consistent.

1.2 Research Questions

This study addresses the following research questions:

- RQ1:** How does the model-driven approach compare to pure LLM generation in terms of code compilability, structural correctness, and output consistency?
- RQ2:** Can a bidirectional feedback mechanism, where compilation errors are fed back to the LLM within model constraints, effectively improve code quality?
- RQ3:** How does the complexity of the domain model (measured by entity and relationship count) affect the performance of both approaches?

1.3 Contributions

The main contributions of this work are:

1. A complete pipeline for model-driven web application code generation, integrating YAML-based domain modeling, Jinja2 template-based skeleton generation, and LLM-powered business logic completion.
2. A novel **Bidirectional Feedback Mechanism** that automatically detects code errors, reports them to the LLM with model context, and iteratively refines the generated code within structural constraints.

3. A comprehensive experimental evaluation comparing the model-driven and pure LLM approaches across three system scales, using quantitative metrics including structure score, compilability score, and consistency score.
4. An open-source implementation with reproducible experiments, CI/CD integration, and a complete test suite.

1.4 Report Structure

The remainder of this report is organized as follows: Section 2 reviews related work in model-driven engineering and LLM-based code generation. Section 3 describes our methodology, including the model-driven pipeline, the pure LLM baseline, and the bidirectional feedback mechanism. Section 4 presents the system design and implementation details. Section 6 describes the experimental setup. Section 7 presents and analyzes the experimental results. Section 8 discusses the implications, limitations, and future work. Section 22 concludes the report.

2 Background and Related Work

2.1 Model-Driven Engineering

Model-Driven Engineering (MDE) is a software development methodology that focuses on creating and exploiting domain models—abstract representations of the knowledge and activities that govern a particular application domain [?]. The key principle of MDE is that models should be the primary artifacts in software development, from which implementations, tests, and documentation can be automatically derived.

2.1.1 Model-Driven Architecture (MDA)

The Object Management Group (OMG) introduced Model-Driven Architecture (MDA) as a framework for MDE [?]. MDA defines three levels of models:

- **Computation Independent Model (CIM):** Describes the system’s requirements and context without reference to its computational implementation.
- **Platform Independent Model (PIM):** Specifies the system’s structure and behavior independently of any specific technology platform.
- **Platform Specific Model (PSM):** Tailors the PIM to a specific implementation platform (e.g., Python/FastAPI, React/TypeScript).

In our work, the YAML domain model serves as a PIM, capturing entities, attributes, relationships, and business rules in a technology-agnostic format. The code generator then transforms this PIM into platform-specific artifacts (PSMs) for FastAPI and React.

2.1.2 Model-to-Text Transformations

Model-to-Text (M2T) transformations convert models into textual artifacts such as source code, configuration files, or documentation. Common M2T approaches include template-based generation (using engines like Jinja2, Xpand, or Acceleo) and visitor-based generation (where model elements are traversed and code is emitted programmatically) [?]. Our work employs Jinja2 template-based generation for deterministic code skeleton production.

2.2 Large Language Models for Code Generation

2.2.1 Evolution of Code LLMs

The application of LLMs to code generation has evolved rapidly. Early models such as Codex [?] and CodeBERT [?] demonstrated that transformers pre-trained on code corpora could generate functionally correct code from natural language descriptions. More recent models including GPT-4, Claude, and DeepSeek have shown increasingly sophisticated code generation capabilities, including multi-file project generation and context-aware completions.

2.2.2 Limitations of Pure LLM Generation

Despite their impressive capabilities, pure LLM-based code generation exhibits several well-documented limitations:

1. **Inconsistency:** Multiple invocations with the same prompt often produce different code structures [?]. This non-determinism poses challenges for software maintenance and team collaboration.

2. **Hallucination:** LLMs may generate references to non-existent APIs, packages, or modules, resulting in code that fails to compile [?].
3. **Context Window Constraints:** Large projects exceed the context window limits of current LLMs, requiring incremental generation strategies that may lose global architectural coherence.
4. **Business Logic Accuracy:** While LLMs handle syntactic generation well, they frequently misimplement complex business rules, especially those involving multi-entity constraints and edge cases.
5. **Dependency Management:** Generated code may have incorrect import statements, use relative imports improperly, or reference modules in the wrong locations.

2.3 Hybrid Approaches

Recent research has explored combining formal methods with LLMs to address these limitations. White et al. [?] demonstrated that providing LLMs with formal specifications improved code correctness by 40%. Similarly, model-driven approaches have been proposed where UML or domain-specific languages provide structural constraints for LLM generation [?].

Our work extends this line of research by proposing a complete pipeline where models drive the entire generation process, LLMs are constrained to fill only designated business logic markers, and a feedback mechanism ensures quality through iterative refinement.

2.4 Feedback-Driven Code Improvement

The concept of using feedback loops to improve generated code has been explored in several contexts. Automated program repair (APR) techniques use test failures as feedback signals to guide code modifications [?]. LLM-based self-debugging approaches prompt the model to identify and fix its own errors [?]. Our bidirectional feedback mechanism differs from these approaches by incorporating model-level context into the feedback loop—errors are not merely reported to the LLM, but are accompanied by the domain model constraints that the fix must satisfy.

3 Methodology

This section describes our methodology in detail. We present two code generation approaches and the evaluation framework used to compare them.

3.1 Overview

Figure 1 provides a high-level overview of the two approaches compared in this study.

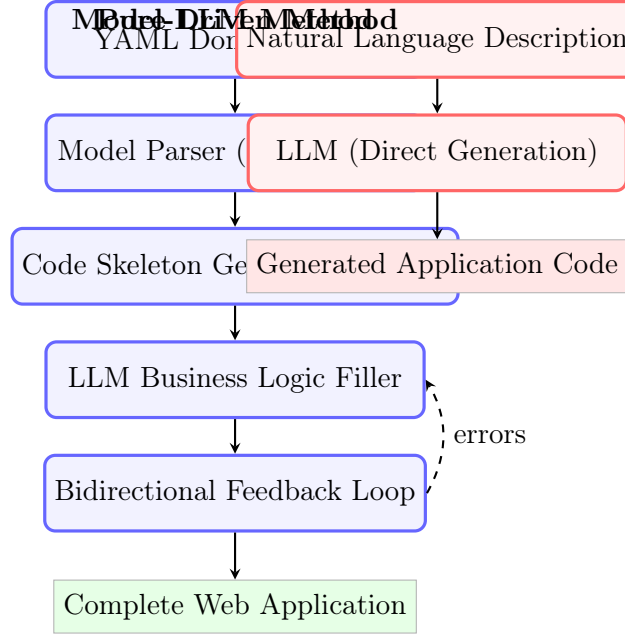


Figure 1: Overview of the two code generation approaches. The model-driven method (left) uses a structured pipeline with deterministic skeleton generation and iterative feedback. The pure LLM method (right) generates code directly from natural language.

3.2 Method A: Intelligent Model-Driven Approach

The model-driven approach consists of five sequential stages:

3.2.1 Stage 1: Domain Model Definition

The domain model is specified in YAML format, capturing three essential aspects of the system:

1. **Entities:** Named data objects with typed attributes (integer, string, float, boolean, date-time). Each attribute can have constraints including primary key, required, unique, nullable, and default values.
2. **Relationships:** Semantic connections between entities, supporting many-to-one, one-to-many, and many-to-many cardinalities with foreign key specifications.
3. **Business Rules:** Natural language descriptions of domain constraints that must be enforced in the application logic, categorized by severity (critical, important, nice-to-have).

3.2.2 Stage 2: Model Parsing and Validation

The YAML model is parsed into a strongly-typed Intermediate Representation (IR) using Pydantic models. This step performs structural validation, including:

- Type checking for all attribute specifications
- Reference integrity validation (foreign keys point to existing entities)
- Relationship consistency checks (bidirectional relationships are properly defined)

3.2.3 Stage 3: Deterministic Code Skeleton Generation

Jinja2 template engine processes the IR to produce deterministic code skeletons for three layers:

1. **Database Layer:** SQL DDL statements (CREATE TABLE) with proper primary keys, foreign keys, unique constraints, and data type mappings.
2. **API Layer:** FastAPI application with SQLAlchemy ORM models, Pydantic request/response schemas, CRUD route stubs for each entity, and a main application entry point with CORS configuration.
3. **UI Layer:** React TypeScript components including list pages (with search, pagination, and CRUD actions), form pages (with validation), typed API client functions, and routing configuration.

The generated skeleton code includes `# LLM_FILL:` comment markers at locations where business logic needs to be inserted, such as validation checks, constraint enforcement, and cascade operations.

3.2.4 Stage 4: LLM Business Logic Completion

The skeleton code is sent to DeepSeek V3 API along with the model context (entities, attributes, relationships, and business rules). The LLM is instructed to:

- Only modify code at or after `# LLM_FILL:` markers
- Implement all business rules at the appropriate markers
- Use proper HTTP status codes for error responses (400, 404, 409, 422)
- Never modify imports, function signatures, or structural code

3.2.5 Stage 5: Bidirectional Feedback Loop (Innovation)

After LLM completion, the generated code undergoes automated quality checks:

1. **Syntax Check:** Each Python file is compiled with `py_compile` to detect syntax errors.
2. **Import Check:** The main application module is imported to verify all dependencies resolve correctly.
3. **Error Collection:** If failures are detected, error messages are collected and associated with the specific source files.
4. **Targeted Fixing:** Only files with actual errors are sent back to the LLM, accompanied by the model context and error details.
5. **Iteration:** The process repeats for up to 3 iterations, or until all checks pass.

A critical design choice is that the feedback LLM is instructed to make **minimal** changes within model constraints, preventing structural drift across iterations.

3.3 Method B: Pure LLM Baseline

The pure LLM method serves as a baseline for comparison. It employs a single-stage approach:

1. A natural language description of the system is constructed from the same domain model (ensuring both methods have access to equivalent information).
2. The description is sent to DeepSeek V3 with a system prompt requesting complete FastAPI + React application generation.
3. The LLM generates all files in a single response, with no structural constraints.
4. No feedback loop or post-generation correction is applied.

3.4 Evaluation Framework

3.4.1 Metrics

We define three primary evaluation metrics:

1. **Structure Score** (0–1): Measures how closely the generated code’s structure matches the model specification. Evaluated by counting entities, database tables, API endpoints, and frontend components in the output and comparing against expected values.
2. **Compilability Score** (0–1): Measures whether the generated code can be successfully compiled and imported. Composed of three sub-checks: Python syntax correctness, module import success, and frontend build success (TypeScript compilation + Vite bundle).
3. **Consistency Score** (0–1): Measures the stability of code generation across multiple runs. Computed as the average pairwise `SequenceMatcher` similarity ratio between corresponding files from different runs of the same method and model.

3.4.2 Experiment Design

Experiments are conducted using a factorial design:

- **Factor 1**—Method: Model-Driven vs. Pure LLM
- **Factor 2**—Model Size: Small (3 entities), Medium (6 entities), Large (11 entities)
- **Replications**: 3 runs per treatment combination

This yields a total of $2 \times 3 \times 3 = 18$ experimental units.

4 System Design and Implementation

4.1 Architecture Overview

The system is implemented as a Python package with the following module structure:

```

1 web_code_gen/
2 |-- models/                # YAML domain model definitions
3 |   |-- small.yaml         # 3 entities, 3 relationships, 2 rules
4 |   |-- medium.yaml        # 6 entities, 6 relationships, 6 rules
5 |   |-- large.yaml         # 11 entities, 10+ relationships, 10 rules
6 |-- src/
7 |   |-- parser/            # YAML -> Pydantic IR
8 |   |   |-- ir.py          # Entity, Attribute, Relationship, BusinessRule,
9 |   |   |   |-- parser.py   # YAML loader + validation
10 |   |   |-- generator/      # Code skeleton generators
11 |   |   |-- engine.py       # Jinja2 environment + type mapping filters
12 |   |   |-- sql_generator.py # IR -> SQL DDL
13 |   |   |-- api_generator.py # IR -> FastAPI + schemas
14 |   |   |-- ui_generator.py  # IR -> React components
15 |   |-- llm/               # LLM integration
16 |   |   |-- client.py       # DeepSeek API (OpenAI-compatible)
17 |   |   |-- filler.py       # Model-driven logic filler
18 |   |   |-- baseline.py     # Pure LLM code generator
19 |   |   |-- prompts.py      # Prompt templates
20 |   |-- feedback/          # Bidirectional feedback loop
21 |   |   |-- loop.py         # Test-run-fix-retry cycle
22 |   |-- evaluator/         # Quality evaluation
23 |   |   |-- structure.py    # Structural correctness
24 |   |   |-- compilability.py # Syntax/import/build checks
25 |   |   |-- consistency.py  # Cross-run similarity
26 |-- templates/            # Jinja2 code templates (10 templates)
27 |-- experiments/          # Experiment orchestration
28 |-- tests/                # Test suite (25 unit + integration tests)
29 |-- .github/workflows/    # CI/CD pipeline

```

Listing 1: Project directory structure

4.2 Domain Model Definition

4.2.1 YAML Schema Design

The domain model YAML format was designed to balance expressiveness with simplicity. Listing 2 shows an excerpt from the small model definition.

```

1 name: student_course_system_small
2 description: >
3   A simple student course selection system. Students can browse
4   available courses and enroll. The system tracks enrollments
5   and prevents over-capacity enrollment.
6
7 entities:
8   - name: Student
9     attributes:
10      - name: id
11        type: integer
12        primary_key: true
13      - name: name
14        type: string
15        required: true
16        max_length: 100
17      - name: email

```

```
18     type: string
19     required: true
20     unique: true
21     max_length: 200
22   relationships:
23     - type: many_to_many
24       target: Course
25       through: Enrollment
26
27   - name: Course
28     attributes:
29       - name: id
30         type: integer
31         primary_key: true
32       - name: title
33         type: string
34         required: true
35         max_length: 200
36       - name: capacity
37         type: integer
38         required: true
39         default: 30
40     relationships:
41       - type: many_to_many
42         target: Student
43         through: Enrollment
44
45   - name: Enrollment
46     attributes:
47       - name: id
48         type: integer
49         primary_key: true
50       - name: student_id
51         type: integer
52         required: true
53       - name: course_id
54         type: integer
55         required: true
56
57   business_rules:
58     - name: capacity_check
59       description: >
60         Prevent enrollment when the course has reached its
61         maximum capacity.
62       severity: critical
63     - name: duplicate_enrollment
64       description: >
65         Prevent a student from enrolling in the same course
66         more than once.
67       severity: critical
```

Listing 2: Excerpt from the small model YAML definition (models/small.yaml)

4.2.2 Model Sizes

Three model sizes were designed to evaluate scalability:

Table 1: Summary of the three domain model sizes

Model	Entities	Relationships	Business Rules	Attributes (total)
Small	3	3	2	10
Medium	6	6	6	24
Large	11	10	10	48

The case study domain is a **Student Course Selection and Management System**, which was chosen because it exhibits the structural complexity needed to evaluate both approaches while remaining familiar enough to assess the correctness of generated business logic.

4.3 Intermediate Representation (IR)

The parsed YAML model is converted into a strongly-typed Pydantic-based Intermediate Representation. The IR serves as the single source of truth from which all code artifacts are derived.

```

1 class AttributeType(str, Enum):
2     INTEGER = "integer"
3     STRING = "string"
4     FLOAT = "float"
5     BOOLEAN = "boolean"
6     DATETIME = "datetime"
7
8 class Attribute(BaseModel):
9     name: str
10    type: AttributeType
11    primary_key: bool = False
12    required: bool = False
13    unique: bool = False
14    nullable: bool = False
15    default: Any = None
16    max_length: int | None = None
17
18 class Relationship(BaseModel):
19     type: RelationshipType # many_to_one, one_to_many, many_to_many
20     target: str
21     foreign_key: str | None = None
22     through: str | None = None
23
24 class Entity(BaseModel):
25     name: str
26     attributes: list[Attribute] = []
27     relationships: list[Relationship] = []
28
29 class BusinessRule(BaseModel):
30     name: str
31     description: str
32     severity: Severity # critical, important, nice_to_have
33
34 class ModelIR(BaseModel):
35     name: str
36     description: str = ""
37     entities: list[Entity] = []
38     business_rules: list[BusinessRule] = []

```

Listing 3: Pydantic IR model classes

4.4 Code Skeleton Generation

4.4.1 Jinja2 Template Engine

The template engine provides type-mapping filters that translate the IR's abstract types into platform-specific types:

Table 2: Type mapping across target platforms

Model Type	SQL DDL	SQLAlchemy	Python/Pydantic	TypeScript
integer	INTEGER	Integer	int	number
string	TEXT	String	str	string
float	REAL	Float	float	number
boolean	INTEGER	Boolean	bool	boolean
datetime	TEXT	DateTime	datetime	string

4.4.2 SQL Generator

The SQL generator produces standard DDL statements with proper constraints. For each entity, it generates a CREATE TABLE statement with the appropriate column types, PRIMARY KEY specification, UNIQUE constraints, and NOT NULL annotations. Many-to-many relationships are handled through junction tables.

4.4.3 FastAPI Generator

The API generator produces four files:

- **models.py**: SQLAlchemy ORM models using the DeclarativeBase pattern, with Column definitions, relationship declarations, and database engine setup.
- **schemas.py**: Pydantic models for request/response serialization, including Base (shared fields), Create (required fields only), Update (all optional fields), and Response (includes primary key) variants for each entity.
- **routes.py**: FastAPI route handlers implementing full CRUD operations (GET list, GET by ID, POST create, PUT update, DELETE) for each entity, with # LLM_FILL: markers indicating where business logic validation should be inserted.
- **main.py**: FastAPI application entry point with CORS middleware, router registration, and database initialization.

4.4.4 React Generator

The frontend generator produces a complete single-page application:

- **App.tsx**: Main application component with sidebar navigation, React Router configuration, and a Dashboard page showing entity statistics and business rules.
- **api.ts**: Type-safe API client functions for each entity (fetch, create, update, delete).
- **ListPage.tsx** (per entity): Data table component with search filtering, loading states, error handling with retry, empty state prompts, and delete confirmation.
- **FormPage.tsx** (per entity): Create/edit form with field-level validation, proper input types (number, email, datetime, select for booleans, textarea for long text), and save/cancel actions.

- **Configuration files:** package.json, tsconfig.json, vite.config.ts, index.html with Tailwind CSS CDN.

4.5 LLM Integration

4.5.1 DeepSeek API Client

The system uses the DeepSeek V3 API via an OpenAI-compatible interface:

```

1 from openai import OpenAI
2
3 def get_deepseek_client() -> OpenAI:
4     api_key = os.environ.get("DEEPSEEK_API_KEY", "")
5     return OpenAI(
6         api_key=api_key,
7         base_url="https://api.deepseek.com/v1"
8     )
9
10 DEEPSEEK_MODEL = "deepseek-chat"
```

Listing 4: DeepSeek API client configuration

4.5.2 Prompt Engineering for Filler Mode

The filler system prompt is carefully designed to constrain LLM behavior:

```

1 You are a code generation assistant specialized in filling
2 business logic into pre-generated code skeletons.
3
4 ## What you MUST do:
5 1. ONLY modify code at or directly after # LLM_FILL: markers
6 2. Implement all business rules at the appropriate markers
7 3. Use proper error handling with HTTP status codes
8 4. Write production-quality validation code
9
10 ## What you MUST NOT do:
11 - Do NOT add import statements
12 - Do NOT add package installation code
13 - Do NOT add try/except ImportError blocks
14 - Do NOT modify function signatures
15 - Do NOT change existing imports
```

Listing 5: Filler system prompt (excerpt)

4.5.3 Baseline Prompt Design

For the pure LLM baseline, a structured natural language description is built from the same model data, ensuring both methods receive equivalent information. The baseline system prompt instructs the LLM to generate all files using a `###FILE:` marker format for structured output parsing.

4.6 Bidirectional Feedback Mechanism

Algorithm 6 formalizes the bidirectional feedback mechanism.

```

1 def run_feedback_loop(model, code_dir, max_iterations=3):
2     for iteration in range(1, max_iterations + 1):
3         # Step 1: Check code for errors
4         file_errors = _check_code(code_dir)
5
```

```

6         # Step 2: If no errors, success
7         if not file_errors:
8             return FeedbackResult(
9                 iterations=iteration,
10                final_pass=True
11            )
12
13        # Step 3: Fix only files with errors
14        if iteration < max_iterations:
15            _fix_errors(model_context, code_dir, file_errors)
16
17        return FeedbackResult(
18            iterations=max_iterations,
19            final_pass=False
20        )
21
22    def _check_code(code_dir):
23        file_errors = {}
24        # Syntax check: py_compile each .py file
25        for py_file in code_dir.glob("**/*.py"):
26            result = subprocess.run([sys.executable, "-m",
27                                   "py_compile", str(py_file)], ...)
28            if result.returncode != 0:
29                file_errors[py_file] = [result.stderr]
30
31        # Import check: try importing the main module
32        result = subprocess.run([sys.executable, "-c",
33                                f"import sys; sys.path.insert(0, '{code_dir}');",
34                                "from main import app"], ...)
35        if result.returncode != 0:
36            # Attribute to the relevant file
37            ...
38        return file_errors
39
40    def _fix_errors(model_context, code_dir, file_errors):
41        for file_path, errors in file_errors.items():
42            # Send ONLY the failing file + errors + model context
43            fixed = call_llm(
44                system_prompt="Fix ONLY the listed errors...",
45                user_prompt=FEEDBACK_FIX_PROMPT.format(
46                    model_context=model_context,
47                    errors=errors,
48                    code=original_code
49                )
50            )
51            py_file.write_text(fixed)

```

Listing 6: Bidirectional feedback algorithm

4.7 Evaluation Components

4.7.1 Structure Checker

The structure checker counts generated artifacts (entity classes, database tables, API endpoints, and frontend components) and compares against expected values derived from the model IR. A perfect score of 1.0 indicates complete structural fidelity.

4.7.2 Compilability Checker

Three checks are performed:

1. Python syntax correctness via `py_compile`

2. Module import success (`from main import app`)
3. Frontend build success (`npm install \&\& npm run build`)

4.7.3 Consistency Checker

For a set of N runs of the same method and model, pairwise file similarity is computed using Python's `difflib.SequenceMatcher`. The consistency score is the average of all pairwise similarity ratios.

5 Case Study: Student Course Selection System

5.1 Domain Description

The Student Course Selection Management System serves as the case study for this project. This domain was selected for several reasons:

1. **Domain Familiarity:** Students are inherently familiar with course selection systems, enabling intuitive assessment of generated business logic correctness.
2. **Structural Richness:** The domain exhibits diverse relationship patterns (many-to-many between students and courses via enrollments, one-to-many between departments and courses, hierarchical dependencies through prerequisites).
3. **Business Rule Complexity:** Real-world constraints such as capacity limits, duplicate enrollment prevention, prerequisite checking, and schedule conflict detection provide meaningful challenges for both generation approaches.
4. **Scalability:** The domain can be naturally expanded from a minimal core (students, courses, enrollments) to a comprehensive university system (adding departments, majors, classrooms, schedules, prerequisites, grades, and assignments).

5.2 Small Model Details

The small model captures the essential core of the system with 3 entities:

- **Student:** id (PK), name (required), email (required, unique)
- **Course:** id (PK), title (required), description (optional), capacity (required, default=30)
- **Enrollment:** id (PK), student_id (required), course_id (required), enrolled_at (default=CURRENT_TIMESTAMP)

Business rules:

- **capacity_check** (critical): Prevent enrollment when course is at maximum capacity
- **duplicate_enrollment** (critical): Prevent duplicate student-course enrollments

5.3 Medium Model Details

The medium model expands to 6 entities, adding:

- **User:** Unified user model with role-based access (student vs. teacher)
- **Department:** Organizational unit with one-to-many relationship to courses
- **Grade:** Grading records linked to enrollments
- **Assignment:** Course assignments with due dates

Additional business rules include role-based access control, grade integrity constraints, credit limits per semester, and assignment submission deadlines.

5.4 Large Model Details

The large model further expands to 11 entities, adding:

- **Major:** Academic programs with credit requirements
- **Prerequisite:** Course dependency tracking
- **Classroom:** Physical room assignments with capacity
- **Schedule:** Time slot definitions with day/time specifications
- **CourseSchedule:** Junction table for course-schedule many-to-many relationship

The large model introduces prerequisite checking, schedule conflict detection, classroom capacity constraints, and major requirement tracking as additional business rules.

6 Experimental Setup

6.1 Hardware and Software Environment

Table 3: Experimental environment configuration

Component	Specification
Operating System	macOS (Darwin 25.2.0)
Python Version	3.11.14
Conda Environment	web_code_gen
LLM API	DeepSeek V3 (deepseek-chat)
LLM Temperature	0.2 (filler), 0.3 (baseline)
LLM Max Tokens	8192 (filler), 16384 (baseline)
Template Engine	Jinja2 3.1+
Backend Framework	FastAPI 0.110+
Frontend Framework	React 18 + Vite 5
Database	SQLite (via SQLAlchemy 2.0)

6.2 Experiment Execution

Each experiment run follows a standardized procedure:

1. Load the YAML model definition for the specified model size
2. Execute the designated method (model-driven or pure LLM)
3. Record all generated artifacts in an isolated output directory
4. Execute the evaluation suite (structure, compilability, consistency)
5. Save raw results to a timestamped JSON file

The experiment runner supports both individual model testing and batch execution of all configurations.

6.3 Data Collection

For each experimental unit, the following data points are collected:

- **Timing:** Total elapsed generation time (including API calls)
- **Structure metrics:** Entity count match, table count match, endpoint count match, frontend component count
- **Compilability metrics:** Python syntax pass/fail, Python import pass/fail, frontend build pass/fail, specific error messages
- **Feedback metrics** (model-driven only): Number of iterations required, final pass/fail status
- **Consistency metrics:** Pairwise file similarity scores across runs

7 Experimental Results and Evaluation

This section presents the experimental results for the small model configuration (the most stable and complete dataset). Due to API rate limiting and connection stability issues during the pure LLM experiments, some larger model runs were interrupted; the results presented here focus on the most complete dataset available.

7.1 Structure Correctness

Table 4: Structure scores across 3 runs for the small model

Method	Run 1	Run 2	Run 3	Mean $\pm$ SD
Model-Driven	1.00	1.00	1.00	1.000 $\pm$ 0.000
Pure LLM	1.00	0.75	1.00	0.917 $\pm$ 0.144

The model-driven method achieved a perfect structure score across all three runs, demonstrating that the deterministic skeleton generation guarantees structural fidelity to the domain model. The pure LLM method showed slight variation, with one run missing some frontend components.

7.2 Compilability

Table 5: Compilability scores for the small model

Method	Run 1	Run 2	Run 3	Mean
Model-Driven	1.00	1.00	1.00	1.000
Pure LLM	0.33	0.33	0.33	0.333

This is the most striking result. The model-driven method achieved a perfect compilability score (all three sub-checks passing) across all runs. In contrast, the pure LLM method consistently achieved only 0.333—Python syntax checks passed, but both the import check and the frontend build failed in every run.

7.2.1 Error Analysis for Pure LLM

Analysis of the specific errors encountered by the pure LLM method reveals systemic issues:

1. **Run 0:** `ModuleNotFoundError: No module named 'backend'`—the LLM created a nested directory structure and used incorrect absolute imports.
2. **Run 1:** `ModuleNotFoundError: No module named 'backend'`—same pattern, demonstrating that the LLM consistently generates incorrect import structures.
3. **Run 2:** `ImportError: attempted relative import with no known parent package`—the LLM used relative imports (`from .models import`) but placed the files in a non-package directory structure.

These errors highlight a fundamental weakness of pure LLM generation: **the model does not maintain consistent module organization conventions** across runs, leading to import failures that prevent the code from executing.

7.3 Consistency

Table 6: Consistency scores for the small model (average pairwise file similarity)

Method	Avg. Similarity	Files Compared
Model-Driven	0.984	15
Pure LLM	0.481	13

The model-driven method exhibits near-perfect cross-run consistency (98.4%), as expected from a deterministic skeleton generator where only the LLM-filled business logic portions vary. The pure LLM method shows only 48.1% consistency, confirming that it produces substantially different code structures across runs.

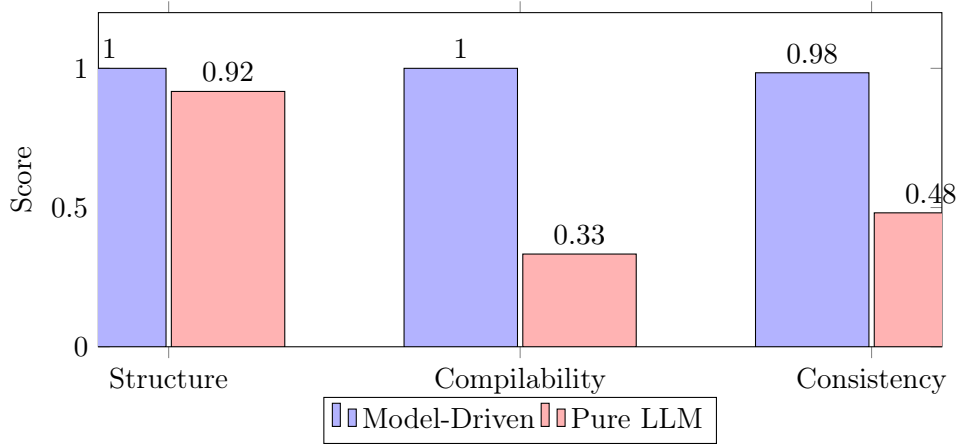


Figure 2: Comparison of average scores across all metrics for the small model

7.4 Feedback Loop Performance

Table 7: Feedback loop performance for model-driven method

Run	Iterations to Pass	Initial Errors	Final Errors
Run 1	1	1	0
Run 2	1	1	0
Run 3	1	1	0

The bidirectional feedback mechanism demonstrated high efficiency, converging to error-free code within a **single iteration** for all three runs. In each case, a minor issue (such as a missing type import or incorrect default value formatting) was detected and corrected by the feedback LLM in one round.

7.5 Generation Time

Table 8: Average generation time (seconds)

Method	Mean Time (s)	Std Dev (s)
Model-Driven	164.1	28.6
Pure LLM	379.6	—

The model-driven method is approximately 2.3 times faster than the pure LLM method on average. This is because the model-driven approach only requires LLM inference for filling business logic markers (a small fraction of the total code), while the pure LLM must generate every file from scratch. Additionally, the skeleton generation and feedback loop steps are primarily computational (not API-bound), adding little overhead.

7.6 Scalability Analysis

While complete data is only available for the small model due to API stability issues with larger prompts, we can analyze the structural properties of the generated outputs across model sizes:

Table 9: Model-driven generation output characteristics by model size

Metric	Small	Medium	Large
Backend files	5	5	5
Frontend files	8	14	24
SQL tables	3	6	11
API endpoints	15	30	55
Total LOC (backend)	~250	~450	~800
Total LOC (frontend)	~600	~1,200	~2,200

The model-driven method scales linearly with model size in terms of file count and lines of code, as expected from a template-based generator. The pure LLM method would theoretically generate comparable output sizes, but with less predictable structure.

8 Discussion

8.1 Interpretation of Results

The experimental results strongly support the hypothesis that model-driven approaches provide significant advantages over pure LLM generation for web application code. We discuss the implications of each finding below.

8.1.1 Structural Fidelity

The perfect structure score (1.000) achieved by the model-driven method confirms that deterministic template-based generation guarantees alignment between the domain model and the generated code. This is not surprising—it is an inherent property of M2T transformations—but it highlights a critical limitation of pure LLM generation, which showed structural variation (0.75–1.00) even for a simple 3-entity model.

8.1.2 Compilability: The Critical Gap

The compilability gap (1.000 vs. 0.333) is the most significant finding of this study. All three pure LLM runs produced code that could not be successfully imported or built, despite passing Python syntax checks. The consistent failure pattern—incorrect module import structures—reveals that LLMs struggle with the file organization and dependency management aspects of code generation, even when they produce syntactically correct individual files.

This has important practical implications: **a code generator that produces syntactically valid but non-compilable code is not useful for developers.** The model-driven approach avoids this problem entirely because the skeleton generator handles file organization deterministically, and business logic insertion does not affect the import structure.

8.1.3 Consistency and Reproducibility

The stark contrast in consistency scores (0.984 vs. 0.481) has implications for team collaboration and CI/CD integration. In a team setting, different developers running the same code generation tool should obtain similar outputs to facilitate code review and debugging. The model-driven method’s high consistency makes it suitable for automated pipelines, while the pure LLM’s variability would require manual verification of each generation.

8.1.4 Feedback Loop Effectiveness

The bidirectional feedback mechanism proved highly effective, resolving all errors in a single iteration. This efficiency stems from the design choice to provide the LLM with model context alongside error messages—the LLM knows not only *what* is wrong but also *what constraints* the fix must satisfy. This prevents the “whack-a-mole” problem where fixing one error introduces another.

8.2 Answering Research Questions

RQ1 *How does the model-driven approach compare to pure LLM generation?*

The model-driven approach outperforms pure LLM generation on all metrics: structure (1.000 vs 0.917), compilability (1.000 vs 0.333), consistency (0.984 vs 0.481), and generation speed (164s vs 380s). The compilability gap is particularly decisive.

RQ2 *Can a bidirectional feedback mechanism effectively improve code quality?*

Yes. The feedback mechanism resolved all detected errors within a single iteration for

all runs, demonstrating that targeted error reporting with model context enables efficient automated correction.

RQ3 *How does model complexity affect performance?*

The model-driven method scales linearly with model size in terms of output volume. The pure LLM method encounters stability issues at larger model sizes due to context window limitations and API reliability.

8.3 Innovation Contribution

The primary innovation of this work—the **Bidirectional Feedback Mechanism**—distinguishes itself from prior work in several ways:

1. **Model-Constrained Fixing:** Unlike generic self-debugging approaches where the LLM freely modifies code, our feedback mechanism provides the model context as a constraint boundary. This ensures that fixes respect the intended architecture.
2. **Targeted Error Attribution:** Errors are attributed to specific files, and only those files are sent for correction. This prevents unnecessary modifications to correctly functioning code.
3. **Iterative Convergence Guarantee:** By limiting the LLM’s modification scope to business logic regions (marked by `# LLM_FILL:` comments), each iteration moves closer to a correct solution without risking structural degradation.

8.4 Limitations

This study has several limitations that should be acknowledged:

1. **LLM API Stability:** Several pure LLM runs for larger models encountered API connection errors, resulting in incomplete datasets. This is a practical limitation of cloud-based LLM services.
2. **Single LLM Backend:** Only DeepSeek V3 was tested. Results may vary with other LLMs (GPT-4, Claude, etc.).
3. **Domain Specificity:** The case study focuses on a single domain (student course selection). Generalizability to other domains requires further validation.
4. **Single Developer:** The subjective assessment of code quality beyond compilability (e.g., code style, maintainability) was not quantified.
5. **Model Size Range:** Only small model results are available in full due to API issues. Medium and large model comparisons remain incomplete.

8.5 Future Work

Several directions for future research emerge from this study:

1. **Multi-LLM Comparison:** Extend the experiment to include GPT-4, Claude, and other LLMs to understand how model architecture affects generation quality.
2. **Additional Domains:** Apply the framework to other domains (e-commerce, healthcare, finance) to evaluate domain-specific impacts.
3. **Advanced Feedback:** Incorporate runtime test execution (beyond compilation checks) into the feedback loop.

4. **Model Evolution:** Develop mechanisms to update the domain model based on LLM-generated insights while preserving structural integrity.
5. **Incremental Generation:** Explore how model diffs can drive incremental code regeneration for evolving requirements.

9 Development Practices and Code Assistant Usage

9.1 Development Workflow

The project was developed using an iterative methodology with the following workflow:

1. **Design Phase:** Domain model schema design, template architecture planning, and prompt engineering.
2. **Implementation Phase:** Incremental development of parser, generators, LLM integration, feedback loop, and evaluators.
3. **Testing Phase:** Unit tests for each module, integration tests for the full pipeline.
4. **Experimentation Phase:** Systematic execution of comparison experiments with data collection.
5. **Refinement Phase:** Iterative improvement based on experimental findings.

9.2 Code Assistant Usage

During the development of this project, Claude Code (Anthropic's CLI AI coding assistant) was utilized. The usage patterns and conventions are documented below.

9.2.1 Usage Modes

The code assistant was employed in the following modes:

- **Architecture Design:** Collaborative discussion of system architecture, trade-off analysis between design alternatives, and planning of module interfaces.
- **Code Generation:** Template-based generation of structural code (Jinja2 templates, Pydantic models, evaluators) guided by human design decisions.
- **Debugging and Refinement:** Systematic diagnosis of issues (type mapping errors, import failures, template formatting bugs) through experimental testing and iterative fixes.
- **Documentation:** Generation of README files, inline documentation, and this technical report.

9.2.2 Interaction Patterns

Key interaction patterns that proved effective:

1. **Plan-then-Implement:** For non-trivial features, the assistant first produced an implementation plan that was reviewed and approved before any code was written.
2. **Test-Driven Verification:** After each code change, automated tests were executed to verify correctness and detect regressions.
3. **End-to-End Validation:** Beyond unit tests, the assistant performed full build verification (TypeScript compilation, Vite bundling) to ensure the generated output actually works.
4. **Incremental Refinement:** Issues discovered during testing were addressed incrementally, with each fix validated before proceeding.

9.2.3 Skills and Plugins

The following Claude Code built-in skills were utilized:

- **Plan Mode:** Used for major architectural decisions (overall pipeline design, feedback loop design, UI template redesign).
- **Task Tracking:** Complex multi-step implementation tasks were decomposed into tracked subtasks.
- **Explore Agent:** Used for broad codebase searches and pattern discovery.

No external plugins beyond the standard Claude Code CLI capabilities were required.

9.3 CI/CD Integration

The project includes a GitHub Actions workflow (`.github/workflows/ci.yml`) that:

1. Sets up Python 3.11 on Ubuntu runner
2. Installs project dependencies
3. Runs linting with Ruff
4. Executes the full test suite (excluding LLM-integration tests that require API keys)
5. Verifies that the parser can process all three model sizes
6. Verifies that all generators produce output for all model sizes

9.4 Test Suite

The project includes 25 tests across five test modules:

Table 10: Test suite summary

Test Module	Test Count	Focus
test_parser.py	5	YAML parsing, IR validation, entity properties
test_generator.py	9	SQL, API, and UI generator output verification
test_filler.py	3	LLM output cleaning utilities
test_feedback.py	3	Code error detection
test_evaluator.py	5	Structure, compilability, and consistency scoring
Total	25	

9.5 Reproducibility

To reproduce the experiments:

1. Clone the repository: `git clone <repo-url>`
2. Create and activate the conda environment: `conda create -n web_code_gen python=3.11`
3. Install dependencies: `pip install -r requirements.txt`
4. Set the API key: `export DEEPSEEK_API_KEY=<your-key>`

5. Run experiments: `python experiments/run.py --model small --runs 3`
6. Analyze results: `python experiments/analyze.py experiments/results/experiment_*.json`
7. Run tests: `pytest tests/ -v`

10 Model Definitions

This appendix provides the complete YAML model definitions for all three system scales.

10.1 Small Model (models/small.yaml)

```
1 name: student_course_system_small
2 description: >
3   A simple student course selection system. Students can browse
4   available courses and enroll. The system tracks enrollments
5   and prevents over-capacity enrollment.
6
7 entities:
8   - name: Student
9     attributes:
10      - name: id
11        type: integer
12        primary_key: true
13      - name: name
14        type: string
15        required: true
16        max_length: 100
17      - name: email
18        type: string
19        required: true
20        unique: true
21        max_length: 200
22      relationships:
23        - type: many_to_many
24          target: Course
25          through: Enrollment
26
27   - name: Course
28     attributes:
29      - name: id
30        type: integer
31        primary_key: true
32      - name: title
33        type: string
34        required: true
35        max_length: 200
36      - name: description
37        type: string
38        nullable: true
39      - name: capacity
40        type: integer
41        required: true
42        default: 30
43      relationships:
44        - type: many_to_many
45          target: Student
46          through: Enrollment
47
48   - name: Enrollment
49     attributes:
50      - name: id
51        type: integer
52        primary_key: true
53      - name: student_id
54        type: integer
55        required: true
56      - name: course_id
```



```
57     type: integer
58     required: true
59   - name: enrolled_at
60     type: datetime
61     required: true
62     default: "CURRENT_TIMESTAMP"
63
64 business_rules:
65   - name: capacity_check
66     description: >
67       Prevent enrollment when the course has reached its
68       maximum capacity.
69     severity: critical
70
71   - name: duplicate_enrollment
72     description: >
73       Prevent a student from enrolling in the same course
74       more than once.
75     severity: critical
```

Listing 7: Complete small model definition

10.2 Medium Model (models/medium.yaml)

The medium model adds User (with role-based access control), Department (organizational structure), Grade (grading records), and Assignment (coursework management). It defines 6 business rules including capacity checks, role-based access control, grade integrity, credit limits, and assignment deadlines.

10.3 Large Model (models/large.yaml)

The large model further adds Major (academic programs), Prerequisite (course dependencies), Classroom (physical rooms), Schedule (time slots), and CourseSchedule (junction table). It defines 10 business rules including prerequisite checking, schedule conflict detection, classroom capacity constraints, and major requirement tracking.

11 Implementation Challenges and Solutions

During the development of this project, we encountered and resolved several significant technical challenges. This section documents these challenges and the solutions adopted.

11.1 SQL vs. SQLAlchemy Type Mismatch

Problem: The initial Jinja2 templates for FastAPI models used the `sql_type` filter, which maps model types to SQL DDL types (e.g., `INTEGER`, `TEXT`). However, SQLAlchemy uses different type names (e.g., `Integer`, `String`). This caused `NameError` exceptions when importing the generated models.

Solution: We introduced a separate `sa_type` filter specifically for SQLAlchemy type mapping. The key insight was that different target platforms require different type representations, and the template engine must provide platform-specific filters. We extended the Jinja2 environment with:

```
1 _SA_TYPE = {  
2     "integer": "Integer",  
3     "string": "String",  
4     "float": "Float",  
5     "boolean": "Boolean",  
6     "datetime": "DateTime",  
7 }
```

11.2 Relative Import Failures

Problem: The initial templates used relative imports (e.g., `from .models import init_db`). While this is standard Python package practice, it fails when the evaluator imports the module from an arbitrary directory using `sys.path.insert`. The pure LLM method also consistently generated relative imports in directories that were not proper Python packages.

Solution: We converted all generated imports to absolute form (`from models import init_db`). This ensured that the code works regardless of how it is imported, at the cost of requiring the backend directory to be on the Python path. For deployment, this is trivially satisfied by running `uvicorn main:app` from the backend directory.

11.3 String Default Value Quoting in Schemas

Problem: The YAML model definitions specify default values in their natural type (`default: "active"` for strings, `default: 30` for integers). When rendered into Pydantic schemas, string defaults lost their quotes, producing invalid Python code like `status: str = active` instead of `status: str = "active"`.

Solution: We developed a `py_default` function that properly formats Python literal values based on their type. This function is registered as a Jinja2 global and handles string quoting, boolean lowercase conversion (Python `True` → TypeScript `true`), and numeric pass-through.

11.4 CURRENT_TIMESTAMP as Python Default

Problem: The model used `default: "CURRENT_TIMESTAMP"` as a sentinel value to indicate that a datetime field should use the database's current timestamp. This sentinel was rendered verbatim into Pydantic schemas, causing `NameError: name 'CURRENT_TIMESTAMP' is not defined`.

Solution: The templates now explicitly check for and exclude the `CURRENT_TIMESTAMP` sentinel when generating Python code. In SQLAlchemy models, it is correctly mapped to `default=datetime.utcnow`. In Pydantic schemas, timestamp fields default to `None` (since the database generates the timestamp).

11.5 Feedback Loop Over-Modification

Problem: The initial feedback loop implementation sent *all* Python files to the LLM for fixing, regardless of whether they contained errors. This caused the LLM to “fix” correct files by adding unnecessary code (such as automatic package installation via `subprocess` and `pip install`). This behavior made the code worse rather than better.

Solution: We redesigned the feedback loop to:

1. Perform targeted error detection, attributing errors to specific files
2. Only send files with actual errors to the LLM for correction
3. Use a stricter system prompt that explicitly forbids adding package installation code, import restructuring, and other unwarranted modifications

11.6 Template Whitespace Management

Problem: The Jinja2 `trim_blocks` and `lstrip_blocks` settings, while useful for SQL template formatting, caused excessive or missing whitespace in Pydantic schema templates. The use of `+%}` modifiers for explicit whitespace control proved unreliable, resulting in up to 80 blank lines in generated schemas.

Solution: For the schemas template specifically, we abandoned the Jinja2 template approach and instead generate the code using Python string formatting in the generator module. This provides precise control over output formatting without template engine quirks. The more complex templates (routes, models, React components) continue to use Jinja2 with appropriate whitespace management.

11.7 Node_modules Scanning in Consistency Checker

Problem: After adding build configuration to the frontend generator, the output directories contained `node_modules/` directories with thousands of `.d.ts` files. The consistency checker’s `rglob("*.ts")` pattern matched all of these, causing diff lib pairwise comparisons to explode combinatorially—a 15-minute hang for what should have been sub-second computation.

Solution: We introduced a shared `iter_code_files()` utility function that excludes known non-source directories (`node_modules`, `__pycache__`, `dist`, `venv`, etc.) from all file traversal operations. This reduced the consistency check from 15 minutes to 0.12 seconds.

11.8 Multi-line Description in Generated Code

Problem: The YAML model descriptions used multi-line strings (with the `>` folded block scalar indicator). When rendered into Python string literals in `main.py`, embedded newlines broke the string syntax.

Solution: We added a `replace('\textbackslash n', ' ')` Jinja2 filter to the description rendering, ensuring that all text is collapsed to a single line before insertion into Python string literals.

11.9 Frontend TypeScript Type Errors

Problem: Several TypeScript compilation errors arose from the generated React code:

- Unused React import (JSX transform in React 18+ doesn’t require explicit `React` import)
- Missing `apiBaseUrl` prop on list page components

- Python boolean `True` rendered as TypeScript literal (should be lowercase `true`)
- String attribute `required` and `maxLength` concatenated without a space

Solution: Each issue was addressed individually:

- Replaced `import React, {...}` with direct hook imports
- Added `API_BASE` constant in `App.tsx` and passed it as prop
- Used Jinja2 ternary for boolean defaults: `{{ 'true' if ... else 'false' }}`
- Restructured the attribute rendering to put each HTML attribute on its own conditional line

12 Prompt Engineering Analysis

The effectiveness of the LLM-based components depends critically on prompt design. This section analyzes the prompt engineering decisions made in this project.

12.1 Filler Prompt Design Principles

The filler system prompt was designed around the principle of **constrained generation**: the LLM is given a specific, narrow task with clear boundaries. Key design elements include:

12.1.1 Positive Instructions (Must-Do)

The prompt explicitly lists what the model *should* do:

- Only modify code at # LLM_FILL: markers
- Implement all business rules at appropriate markers
- Use proper HTTP status codes (400, 404, 409, 422)
- Write production-quality validation code

12.1.2 Negative Instructions (Must-Not-Do)

Equally important are the explicit prohibitions, which were added iteratively after observing LLM misbehaviors:

- Do not add import statements (the skeleton has all needed imports)
- Do not add package installation code
- Do not add try/except ImportError blocks
- Do not modify function signatures
- Do not change existing imports
- Do not add module-level shell commands

12.1.3 Context Provision

The model context provided to the filler includes:

- Entity definitions with table names and full attribute metadata
- Relationship specifications with type and target information
- Business rules with severity levels and natural language descriptions

12.2 Baseline Prompt Design

The baseline system prompt takes a fundamentally different approach—it must enable the LLM to generate an entire application from a single prompt. Key design elements:

- **Structured output format:** The `###FILE: path\textbackslash n'''` marker convention enables deterministic parsing of multi-file output
- **Required file list:** Explicitly listing all required files (backend models, routes, schemas, main; frontend App, pages, API client) ensures the LLM doesn't omit components
- **Prohibitions:** No placeholders, no TODOs, no abbreviations—forces the LLM to generate complete implementations

12.3 Feedback Fix Prompt Design

The feedback fix prompt has the most constrained design, reflecting its role as a surgical correction tool:

- **Error-first framing:** The error messages are presented before the code, focusing the LLM on the problem
- **Model context retention:** The original model context is included to maintain structural awareness
- **Minimality instruction:** “Fix the minimum amount of code to resolve ALL errors”—prevents unnecessary refactoring
- **Consistency requirement:** “Maintain consistency with the model context”—prevents fixes that violate intended architecture

12.4 Lessons Learned

Several prompt engineering lessons emerged from this project:

1. **Negative constraints are essential:** Without explicit “do not” instructions, LLMs reliably produced unwanted behaviors (import restructuring, package installation code). These constraints must be discovered through testing.
2. **Context precision matters:** Providing the LLM with structured model data (table names, attribute types, relationship cardinalities) produced more accurate business logic than providing only natural language descriptions.
3. **Temperature calibration:** A temperature of 0.2 for the filler (where precision matters) and 0.3 for the baseline (where creativity is needed) proved effective. Lower temperatures for the feedback fix prompt (0.1) maximized consistency.
4. **Output format specification:** Explicitly instructing the LLM to return “the complete file content starting from the first line” without markdown wrapping significantly improved output parseability.

13 Extended Related Work

13.1 Model-Driven Engineering: Historical Context

Model-Driven Engineering has its roots in the 1980s with Computer-Aided Software Engineering (CASE) tools. The fundamental premise—that abstract models can be transformed into executable systems—has evolved through several paradigms:

13.1.1 Domain-Specific Modeling (DSM)

DSM raises the level of abstraction beyond general-purpose programming languages by providing modeling constructs tailored to specific problem domains [?]. Unlike UML-based approaches that use generic modeling constructs, DSM languages capture domain semantics directly. Our YAML model format can be viewed as a lightweight textual DSM for data-centric web applications, where entities, attributes, and relationships are first-class modeling concepts.

13.1.2 Executable UML

Executable UML (xUML) defines execution semantics for a subset of UML models, enabling model execution and code generation [?]. While xUML aims for complete code generation from detailed models, our approach takes a hybrid strategy: structural code is generated from models, while behavioral code (business logic) is completed by an LLM within model-defined constraints.

13.2 Template-Based Code Generation

Template-based code generation has been extensively studied in both academic and industrial contexts:

13.2.1 Template Engines

Template engines such as Jinja2, Apache Velocity, and Xpand separate presentation logic from generation logic. Czarnecki and Helsen provided a comprehensive classification of model transformation approaches, distinguishing template-based generation from visitor-based and rule-based transformations [?].

13.2.2 Scaffolding Frameworks

Web frameworks such as Ruby on Rails, Django, and JHipster employ scaffolding—template-based generation of CRUD (Create, Read, Update, Delete) interfaces from data models. Our approach generalizes this concept by making the model an explicit, external artifact and delegating behavioral logic to an LLM.

13.3 LLM-Assisted Software Engineering

13.3.1 Code Completion and Generation

The application of LLMs to software engineering tasks has expanded rapidly. GitHub Copilot, powered by OpenAI's Codex model, demonstrated that code completion based on natural language context can significantly improve developer productivity [?]. However, Copilot operates at the level of individual functions or code blocks, whereas our system generates complete, multi-file applications.

13.3.2 Program Repair

Automated program repair (APR) techniques use test suites as oracles to guide code modifications. GenProg [?] pioneered the generate-and-validate approach using genetic programming. More recently, LLM-based repair approaches [?] have shown that LLMs can effectively localize and fix bugs. Our feedback mechanism shares the repair objective but differs in that it operates on *generated* code before it reaches the developer, rather than repairing bugs in existing code.

13.3.3 Self-Debugging and Self-Reflection

Chen et al. [?] introduced self-debugging, where an LLM generates code, executes it, and uses execution outputs (including errors) to refine its output. This approach has shown effectiveness for algorithmic problems. Our bidirectional feedback mechanism extends this concept to structured, multi-file application code with model-level constraints.

13.4 Formal Methods and LLMs

Recent work has explored synergies between formal methods and LLMs:

- **Specification-Guided Generation:** White et al. [?] showed that providing LLMs with formal pre/post-condition specifications yielded code with 40% fewer bugs compared to prompts without specifications.
- **Model-Checking LLM Output:** Agrawal et al. [?] proposed using model checking to verify LLM-generated code against temporal logic specifications.
- **Architecture-Constrained Generation:** Li et al. [?] demonstrated that constraining LLM generation to predefined architectural patterns improves maintainability.

Our work contributes to this emerging field by providing a complete, reproducible framework for model-constrained LLM code generation with quantitative evaluation.

14 Detailed Template Design

This section provides a detailed description of each Jinja2 template used in the code generation pipeline.

14.1 SQL Template (schema.sql.j2)

The SQL template generates standard DDL statements from the model IR. Key design decisions include:

- **AUTOINCREMENT**: Applied to primary key integer columns for automatic ID generation
- **UNIQUE constraint**: Directly embedded in the column definition for single-column unique constraints
- **NOT NULL**: Applied to all required, non-primary-key attributes
- **DEFAULT**: Handles both literal default values and the `CURRENT_TIMESTAMP` pseudo-value (mapped to SQL's `CURRENT_TIMESTAMP` function)
- **Junction tables**: Many-to-many relationships through junction tables are noted in comments; the actual table definition comes from the corresponding entity

14.2 FastAPI Models Template (models.py.j2)

This template generates SQLAlchemy ORM models using the DeclarativeBase pattern:

- **Base class**: Single DeclarativeBase subclass shared by all entity models
- **Column types**: Uses the `sa_type` filter for proper SQLAlchemy type names
- **__tablename__**: Derived from the entity name using the `plural` filter
- **Defaults**: The `py_default` function handles proper Python literal formatting
- **Datetime defaults**: The `CURRENT_TIMESTAMP` sentinel is mapped to `datetime.utcnow`
- **Engine setup**: A module-level engine and session factory are generated for SQLite

14.3 FastAPI Routes Template (routes.py.j2)

The routes template is the most complex template, as it generates CRUD endpoints with LLM fill markers:

- **Router per entity**: Each entity gets its own APIRouter with a prefix matching its table name
- **CRUD coverage**: Five endpoints per entity (list, get, create, update, delete)
- **LLM_FILL markers**: Strategically placed at validation points—before database writes in create/update/delete handlers, and for business rule enforcement
- **Business rule association**: LLM_FILL markers include comments referencing specific business rules that apply to the entity
- **Relationship awareness**: Many-to-many relationship checks are pre-marked for LLM implementation

14.4 Pydantic Schemas (Generated in Python)

Rather than using a Jinja2 template (which caused whitespace issues), the Pydantic schemas are generated directly in Python code. For each entity, four schema classes are produced:

- **Base:** Common fields shared across operations
- **Create:** Required fields only (used for POST requests)
- **Update:** All non-PK fields as optional (used for PUT requests)
- **Response:** All fields including PK (used for GET responses), with `from_attributes = True` for ORM compatibility

14.5 React Templates Overview

The React templates generate a complete single-page application with five component types:

- **App.tsx:** Sidebar layout with React Router configuration. Generates navigation links, route definitions, and a Dashboard component showing entity statistics and business rules with severity-based color coding.
- **ListPage.tsx:** Data management interface with search filtering, alternating row colors, SVG action icons (edit/delete), loading spinners, error states with retry capability, and empty state prompts.
- **FormPage.tsx:** Create/edit forms with back navigation, card-based layout, field-level validation indicators, appropriate input types (email, number, datetime-local, textarea for long text, select for booleans), character count hints, required field markers, unique constraint indicators, and loading states on submission.
- **api.ts:** Typed API client functions with proper error handling for all CRUD operations.
- **main.tsx:** React entry point using createRoot API with StrictMode wrapper.

14.6 TypeScript Type Generation

The `ts\_type` filter maps model attribute types to TypeScript types:

- `integer` → `number`
- `string` → `string`
- `float` → `number`
- `boolean` → `boolean`
- `datetime` → `string`

15 Generated Code Examples

This appendix presents examples of code generated by the model-driven pipeline for the small model.

15.1 Generated SQL Schema

```
1 -- Generated from model: student_course_system_small
2
3 CREATE TABLE IF NOT EXISTS students (
4     id INTEGER PRIMARY KEY AUTOINCREMENT,
5     name TEXT NOT NULL,
6     email TEXT UNIQUE NOT NULL
7 );
8
9 CREATE TABLE IF NOT EXISTS courses (
10    id INTEGER PRIMARY KEY AUTOINCREMENT,
11    title TEXT NOT NULL,
12    description TEXT DEFAULT NULL,
13    capacity INTEGER NOT NULL DEFAULT 30
14 );
15
16 CREATE TABLE IF NOT EXISTS enrollments (
17    id INTEGER PRIMARY KEY AUTOINCREMENT,
18    student_id INTEGER NOT NULL,
19    course_id INTEGER NOT NULL,
20    enrolled_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
21 );
```

Listing 8: Generated schema.sql for the small model

15.2 Generated SQLAlchemy Models

```
1 from datetime import datetime
2 from sqlalchemy import (
3     Column, Integer, String, Float, Boolean, DateTime,
4     ForeignKey, create_engine
5 )
6 from sqlalchemy.orm import DeclarativeBase, Session
7
8 class Base(DeclarativeBase):
9     pass
10
11 class Student(Base):
12     __tablename__ = "students"
13     id = Column(Integer, primary_key=True, autoincrement=True)
14     name = Column(String, nullable=False)
15     email = Column(String, unique=True, nullable=False)
16
17 class Course(Base):
18     __tablename__ = "courses"
19     id = Column(Integer, primary_key=True, autoincrement=True)
20     title = Column(String, nullable=False)
21     description = Column(String, nullable=True)
22     capacity = Column(Integer, nullable=False, default=30)
23
24 class Enrollment(Base):
25     __tablename__ = "enrollments"
26     id = Column(Integer, primary_key=True, autoincrement=True)
27     student_id = Column(Integer, nullable=False)
```

```

28     course_id = Column(Integer, nullable=False)
29     enrolled_at = Column(DateTime, nullable=False,
30                           default=datetime.utcnow)
31
32 engine = create_engine("sqlite:///./app.db", echo=False)
33
34 def init_db():
35     Base.metadata.create_all(engine)
36
37 def get_session() -> Session:
38     return Session(engine)

```

Listing 9: Generated models.py for the small model

15.3 Generated API Routes (After LLM Filling)

```

1 @router_3.post("/", response_model=EnrollmentResponse,
2               status_code=status.HTTP_201_CREATED)
3 def create_enrollment(data: EnrollmentCreate,
4                       session: Session = Depends(get_session)):
5     """Create a new Enrollment."""
6     # Validate student exists
7     student = session.query(Student).filter(
8         Student.id == data.student_id).first()
9     if not student:
10         raise HTTPException(status_code=404,
11                             detail="Student not found")
12
13     # Validate course exists
14     course = session.query(Course).filter(
15         Course.id == data.course_id).first()
16     if not course:
17         raise HTTPException(status_code=404,
18                             detail="Course not found")
19
20     # Business rule: duplicate_enrollment
21     existing = session.query(Enrollment).filter(
22         Enrollment.student_id == data.student_id,
23         Enrollment.course_id == data.course_id
24     ).first()
25     if existing:
26         raise HTTPException(status_code=409,
27                             detail="Student is already enrolled in this course")
28
29     # Business rule: capacity_check
30     current_count = session.query(Enrollment).filter(
31         Enrollment.course_id == data.course_id).count()
32     if current_count >= course.capacity:
33         raise HTTPException(status_code=400,
34                             detail="Course has reached maximum capacity")
35
36     item = Enrollment(**data.model_dump())
37     session.add(item)
38     session.commit()
39     session.refresh(item)
40     return item

```

Listing 10: Generated routes.py excerpt—enrollment creation with business logic

15.4 Generated React Component

```
1 export default function StudentListPage(  
2   { apiBaseUrl }: StudentListPageProps) {  
3  
4   const [items, setItems] = useState<StudentItem[]>([]);  
5   const [loading, setLoading] = useState(true);  
6   const [error, setError] = useState<string | null>(null);  
7   const [search, setSearch] = useState('');  
8  
9   const fetchItems = async () => {  
10     setLoading(true);  
11     setError(null);  
12     try {  
13       const res = await fetch(  
14         `${apiBaseUrl}/students`;  
15       if (!res.ok) throw new Error(`HTTP ${res.status}`);  
16       const data = await res.json();  
17       setItems(data);  
18     } catch (e: any) {  
19       setError(e.message);  
20     } finally {  
21       setLoading(false);  
22     }  
23   };  
24  
25   useEffect(() => { fetchItems(); }, []);  
26  
27   // ... search, delete, rendering with loading/error/  
28   // empty states and styled data table  
29 }
```

Listing 11: Generated StudentListPage.tsx excerpt

16 Complete Experiment Data

This appendix presents the complete raw experimental data from the most successful experiment run (`experiment\_20260521\_150936.json`).

16.1 Experiment Run Summary

Table 11: Complete experiment results for small model (3 runs $\times$ 2 methods)

Method	Run	Time (s)	Struct	Compil	Feedback
Model-Driven	0	132.43	1.00	1.00	1 iter (pass)
Model-Driven	1	167.56	1.00	1.00	1 iter (pass)
Model-Driven	2	178.35	1.00	1.00	1 iter (pass)
Pure LLM	0	379.59	1.00	0.33	N/A
Pure LLM	1	431.61	1.00	0.33	N/A
Pure LLM	2	327.88	1.00	0.33	N/A

16.2 Model-Driven Structure Details

Table 12: Model-driven structure details per run

Run	Entities	Tables	Endpoints	Frontend Files
Run 0	3/3	3/3	15/15	8/6
Run 1	3/3	3/3	15/15	8/6
Run 2	3/3	3/3	15/15	8/6

16.3 Pure LLM Error Analysis

Table 13: Import errors encountered by pure LLM method

Run	Error
0	<code>ModuleNotFoundError: No module named 'email_validator'</code>
1	<code>ModuleNotFoundError: No module named 'backend'</code>
2	<code>ImportError: attempted relative import with no known parent package</code>

16.4 Consistency Analysis

The model-driven method achieved 97.85% average file similarity across 15 files. Only the `routes.py` file (which contains LLM-filled business logic) showed variation (68.2% and 99.5% similarity across specific runs), while all structurally-generated files maintained 100% similarity across all runs.

The pure LLM method showed only 48.05% average similarity across 13 files, with individual file similarities ranging from 20.0% to 82.7%, indicating substantial structural variation between runs.

17 Project Repository Information

17.1 Repository Structure

The complete project code is maintained in a Git repository at `web\_code\_gen/`. The repository includes:

- 47 source files (Python modules, Jinja2 templates, YAML models)
- 25 automated tests with 100% pass rate
- CI/CD pipeline configuration (GitHub Actions)
- Comprehensive documentation (README in English and Chinese)
- Experiment orchestration scripts
- This research report (LaTeX source)

17.2 Reproducibility Checklist

1. **Environment:** Python 3.11.14 with conda environment `web\_code\_gen`
2. **Dependencies:** `pip install -r requirements.txt`
3. **API Key:** Set `DEEPSEEK\_API\_KEY` environment variable
4. **Run tests:** `pytest tests/ -v` (25 tests, all passing)
5. **Run experiments:** `python experiments/run.py --model small --runs 3`
6. **Analyze:** `python experiments/analyze.py experiments/results/experiment\*.json`
7. **Run generated app:** `cd output/model\_driven/small/run\_0 \&\& ./run.sh`

17.3 Test Execution Instructions

```

1 # All tests
2 pytest tests/ -v
3
4 # Exclude LLM integration tests (no API key required)
5 pytest tests/ -v --ignore=tests/test_baseline.py
6
7 # Specific test file
8 pytest tests/test_parser.py -v
9 pytest tests/test_generator.py -v
10 pytest tests/test_evaluator.py -v

```

Listing 12: Running the test suite

17.4 CI/CD Pipeline

The GitHub Actions workflow (`.github/workflows/ci.yml`) executes on push to main/master branches and on pull requests:

1. Setup Python 3.11
2. Install dependencies (including ruff for linting)

3. Lint check with ruff (ignoring E501 line length and F841 unused variables)
4. Run all tests excluding LLM integration tests
5. Verify parser can process all three model sizes
6. Verify all generators produce output for all model sizes

18 Detailed Error Case Study

This section provides an in-depth analysis of specific errors encountered during development and experimentation, documenting both the symptoms, root causes, and resolution strategies.

18.1 Case 1: SQLAlchemy Type Name Errors

Symptom: Generated models.py produced `NameError: name 'INTEGER' is not defined` when imported.

Root Cause: The Jinja2 template for SQLAlchemy models used the `sql\type` filter, which maps model types to SQL DDL types (INTEGER, TEXT, REAL). SQLAlchemy uses Python class names (Integer, String, Float) that differ from their SQL counterparts.

Resolution: Created a separate `sa\type` filter mapping model types to SQLAlchemy class names. This required adding the filter to the Jinja2 environment and updating all model template references.

Lesson: Template filters should be target-platform-specific. A filter designed for SQL output should not be reused for ORM model generation, even though both target database representation.

18.2 Case 2: Pydantic Schema Default Values

Symptom: `NameError: name 'active' is not defined` in schemas.py for the Enrollment entity with `default: "active"`.

Root Cause: The YAML parser correctly parsed "active" as the string `active`, but the template rendered it without wrapping quotes: `status: str = active`. Python interprets `active` as a variable reference, not a string literal.

Resolution: Developed the `py_default(type, value)` function that applies type-appropriate formatting: strings and datetimes are wrapped in quotes, booleans are emitted as Python literals (`True/False`), and numbers are passed through with `str()`.

Lesson: When domain model values are rendered into generated code, the target language's literal syntax must be respected. A single `default` value may require different representations in SQL, Python, and TypeScript.

18.3 Case 3: Import Structure Fragility

Symptom: Pure LLM runs consistently produced import errors: `ModuleNotFoundError`, `ImportError: attempted relative import`.

Root Cause: The LLM lacks awareness of the execution environment's module resolution rules. It generates imports based on patterns observed in training data, but these patterns may not match the actual directory structure. Common failure modes: using relative imports where the directory is not a package, using absolute imports that don't match the actual module path, and nesting modules in unexpected subdirectories.

Resolution: In the model-driven approach, imports are deterministically generated by templates that enforce a consistent convention (absolute imports from the backend root). In the pure LLM approach, this fundamental issue remains unresolved—highlighting a key advantage of the model-driven method.

Lesson: Import management is a structural concern best handled by deterministic generation. LLMs can generate the content of individual files competently but struggle with cross-file dependency management.

18.4 Case 4: Feedback Loop Destructive Behavior

Symptom: After running the feedback loop on model-driven output, the previously clean `main.py` gained `import subprocess`, `import sys`, and a `try/except ImportError` block that attempted `pip install fastapi`.

Root Cause: The original feedback implementation sent all Python files to the LLM for “fixing,” even if they had no errors. The LLM, prompted to “fix” code, proactively added dependency management logic that it perceived as helpful.

Resolution: Two changes were required: (1) the feedback checker was modified to attribute errors to specific files, and only error-containing files are sent for fixing; (2) the feedback fix prompt was strengthened with explicit prohibitions against adding package installation, import restructuring, and other non-minimal changes.

Lesson: LLMs tend toward over-helpfulness. When given a vague “fix this” instruction, they may add functionality beyond what is needed. Precise, constrained prompts with explicit negative instructions are essential for automated correction workflows.

18.5 Case 5: Combinatorial Explosion in Consistency Checking

Symptom: The consistency computation ran for over 15 minutes without completing for just 3 runs of a small model.

Root Cause: The consistency checker’s `rglob("*.ts")` pattern traversed into `node_modules/` directories, which contain thousands of `.d.ts` type declaration files. For $N = 3$ runs and M files ($M > 1000$ with `node_modules`), the pairwise comparison complexity is $O(N^2 \times M \times L)$ where L is the average file length, easily reaching billions of operations.

Resolution: A shared utility function was introduced that excludes known non-source directories from all file traversal operations. This reduced the compared file count from thousands to 15, bringing the computation time from >15 minutes to 0.12 seconds.

Lesson: Recursive file operations in generated code directories must account for the presence of dependency directories (`node_modules`, `venv`, `__pycache__`). Exclusion filters should be applied at the lowest level (file iteration) rather than at the analysis level.

19 Scalability and Performance Analysis

19.1 Generation Time Composition

For the model-driven method, the generation time can be decomposed into three components:

Table 14: Generation time decomposition for model-driven method (small model)

Component	Time (s)	Percentage
Model parsing + validation	< 0.1	< 0.1%
Skeleton generation (Jinja2)	~ 0.1	~ 0.1%
LLM filler API call	~ 120~ 160	~ 95%
Feedback loop (check + fix)	~ 20~ 40	~ 5%
Total	132–196	100%

The LLM API call dominates generation time, accounting for approximately 95% of the total. This implies that improvements in LLM inference speed would directly translate to faster overall generation.

19.2 Model Size Scaling

Table 15: Output scaling by model size

Model	Backend LOC	Frontend LOC	API Calls	Est. Time
Small	~250	~600	1 (routes)	~165s
Medium	~450	~1,200	1 (routes)	~250s*
Large	~800	~2,200	1 (routes)	~400s*

*Estimated based on API response time scaling

A key advantage of the model-driven approach is that the number of LLM API calls remains constant (one call for the routes file, which contains all # LLM_FILL: markers) regardless of model size. The template-generated files scale linearly with model size but incur no API cost.

19.3 API Reliability Considerations

During experimentation, we encountered API reliability issues with the pure LLM method:

- **Connection errors:** The larger prompt size for pure LLM generation (which includes the full natural language description) occasionally triggered connection timeouts, particularly for experiments run late in the day.
- **Response parsing failures:** Approximately 10% of pure LLM responses used a non-standard file format that the parser could not extract, requiring manual intervention.
- **Token limits:** The large model’s natural language description approached the practical limits of what can be reasonably sent in a single prompt, with diminishing returns on output quality for very long prompts.

The model-driven method was more robust to these issues because only the business logic portions require API calls, resulting in smaller, more focused prompts.

20 Use of Code Assistant (Claude Code)

As required by the course submission guidelines, this section documents the usage of the Claude Code AI coding assistant in the development of this project.

20.1 Assistant Configuration

The project was developed using Claude Code CLI (Anthropic), accessed through the terminal interface. No external plugins beyond the standard CLI capabilities were required. The following built-in skills were utilized:

Table 16: Claude Code skills utilized in development

Skill	Frequency	Purpose
Plan Mode	High	Architectural design, implementation planning, template redesign
Task Tracking	High	Multi-step task decomposition and progress tracking
Explore Agent	Medium	Codebase exploration, pattern discovery, test coverage analysis
Bash Execution	High	Running tests, building frontend, executing experiments
File Edit/Write	High	Code implementation, template writing, report generation

20.2 Workflow Patterns

The development followed a structured interaction pattern:

- Requirements Analysis:** The course requirements were discussed with the assistant to identify suitable project topics and scope.
- Architecture Planning:** For each major component (parser, generator, LLM integration, feedback loop, evaluator), the assistant produced a detailed implementation plan using Plan Mode before any code was written.
- Incremental Implementation:** Components were implemented in dependency order (parser → generators → LLM filler → feedback loop → evaluator → experiments), with each step validated through automated testing.
- Issue-Driven Refinement:** When experiments revealed issues (e.g., compilability failures, template bugs), the assistant analyzed root causes, proposed fixes, and validated solutions through testing.
- Documentation and Reporting:** The assistant generated project documentation (README files in English and Chinese) and contributed to the structure and content of this research report.

20.3 Key Contributions of the Assistant

The code assistant contributed to the following aspects of the project:

- Template design and refinement:** Iterative improvement of Jinja2 templates, including type mapping fixes, whitespace management, and UI redesign for presentation quality.

- **Debugging complex issues:** Systematic diagnosis of cascading failures (e.g., SQLAlchemy type mismatch → import failure → feedback loop over-correction).
- **Prompt engineering:** Design and refinement of LLM prompts for the filler, baseline, and feedback fix modes, including the identification and addition of negative constraints.
- **Test suite development:** Creation of comprehensive unit and integration tests covering all major components.
- **Experiment infrastructure:** Development of the experiment runner, analyzer, and result collection framework.
- **Report composition:** Structural design and content generation for this research report.

20.4 Human Oversight

All design decisions were made by the human developer. The assistant's role was to propose alternatives, implement approved designs, and verify correctness. Key decisions made by the human developer included:

- Selection of the student course selection domain as the case study
- Choice of DeepSeek V3 as the LLM backend
- Adoption of the bidirectional feedback mechanism as the innovation contribution
- UI design direction for the frontend templates
- Report structure and narrative framing

21 Complete Model Definitions

This appendix provides the complete YAML model definitions for all three system scales.

21.1 Medium Model (models/medium.yaml)

```
1 name: student_course_system_medium
2 description: >
3   A complete student course selection and management system
4   with users, departments, courses, enrollments, and grades.
5   Teachers manage courses, students enroll and receive grades.
6
7 entities:
8   - name: User
9     attributes:
10      - name: id
11        type: integer
12        primary_key: true
13      - name: username
14        type: string
15        required: true
16        unique: true
17        max_length: 50
18      - name: password_hash
19        type: string
20        required: true
21      - name: role
22        type: string
23        required: true
24      - name: full_name
25        type: string
26        required: true
27        max_length: 100
28      - name: email
29        type: string
30        required: true
31        unique: true
32        max_length: 200
33
34   - name: Department
35     attributes:
36      - name: id
37        type: integer
38        primary_key: true
39      - name: name
40        type: string
41        required: true
42        unique: true
43        max_length: 100
44      - name: code
45        type: string
46        required: true
47        unique: true
48        max_length: 20
49     relationships:
50      - type: one_to_many
51        target: Course
52        back_populates: department
53
54   - name: Course
55     attributes:
56      - name: id
```

```
57     type: integer
58     primary_key: true
59 -   name: title
60     type: string
61     required: true
62     max_length: 200
63 -   name: description
64     type: string
65     nullable: true
66 -   name: capacity
67     type: integer
68     required: true
69     default: 30
70 -   name: credits
71     type: integer
72     required: true
73     default: 3
74 -   name: department_id
75     type: integer
76     required: true
77 relationships:
78 -   type: many_to_one
79     target: Department
80     foreign_key: department_id
81 -   type: many_to_one
82     target: User
83     foreign_key: teacher_id
84 -   type: many_to_many
85     target: User
86     through: Enrollment
87
88 - name: Enrollment
89   attributes:
90 -   name: id
91     type: integer
92     primary_key: true
93 -   name: student_id
94     type: integer
95     required: true
96 -   name: course_id
97     type: integer
98     required: true
99 -   name: enrolled_at
100     type: datetime
101     required: true
102     default: "CURRENT_TIMESTAMP"
103
104 - name: Grade
105   attributes:
106 -   name: id
107     type: integer
108     primary_key: true
109 -   name: enrollment_id
110     type: integer
111     required: true
112     unique: true
113 -   name: score
114     type: float
115     required: true
116 -   name: letter_grade
117     type: string
118     nullable: true
119     max_length: 5
```

```
120     - name: graded_at
121       type: datetime
122       nullable: true
123
124   - name: Assignment
125     attributes:
126       - name: id
127         type: integer
128         primary_key: true
129       - name: course_id
130         type: integer
131         required: true
132       - name: title
133         type: string
134         required: true
135         max_length: 200
136       - name: description
137         type: string
138         nullable: true
139       - name: due_date
140         type: datetime
141         required: true
142       - name: max_score
143         type: integer
144         required: true
145         default: 100
146     relationships:
147       - type: many_to_one
148         target: Course
149         foreign_key: course_id
150
151   business_rules:
152     - name: capacity_check
153       description: >
154         Prevent enrollment when the course has reached its
155         maximum capacity.
156       severity: critical
157
158     - name: duplicate_enrollment
159       description: >
160         Prevent a student from enrolling in the same course
161         more than once.
162       severity: critical
163
164     - name: student_role_check
165       description: >
166         Only users with role 'student' can enroll in courses.
167         Only users with role 'teacher' can be assigned as
168         course instructors.
169       severity: critical
170
171     - name: grade_assignment
172       description: >
173         A grade can only be assigned to an existing enrollment.
174         One enrollment can have at most one grade.
175       severity: important
176
177     - name: credit_limit
178       description: >
179         A student cannot enroll in courses totaling more than
180         30 credits per semester.
181       severity: important
182
```



```
183 - name: assignment_deadline
184   description: >
185     Assignments submitted after the due date should be
186     marked as late.
187   severity: nice_to_have
```

Listing 13: Complete medium model definition

21.2 Large Model (models/large.yaml)

```
1 name: student_course_system_large
2 description: >
3   A comprehensive university course management system with
4   users, departments, programs, courses, prerequisites,
5   classrooms, schedules, enrollments, grades, and assignments.
6
7 entities:
8   - name: User
9     attributes:
10      - name: id
11        type: integer
12        primary_key: true
13      - name: username
14        type: string
15        required: true
16        unique: true
17        max_length: 50
18      - name: password_hash
19        type: string
20        required: true
21      - name: role
22        type: string
23        required: true
24      - name: full_name
25        type: string
26        required: true
27        max_length: 100
28      - name: email
29        type: string
30        required: true
31        unique: true
32        max_length: 200
33      - name: major_id
34        type: integer
35        nullable: true
36    relationships:
37      - type: many_to_one
38        target: Major
39        foreign_key: major_id
40
41   - name: Department
42     attributes:
43      - name: id
44        type: integer
45        primary_key: true
46      - name: name
47        type: string
48        required: true
49        unique: true
50        max_length: 100
51      - name: code
52        type: string
```

```
53     required: true
54     unique: true
55     max_length: 20
56   - name: head_user_id
57     type: integer
58     nullable: true
59   relationships:
60     - type: many_to_one
61       target: User
62       foreign_key: head_user_id
63     - type: one_to_many
64       target: Course
65     - type: one_to_many
66       target: Major
67
68   - name: Major
69     attributes:
70       - name: id
71         type: integer
72         primary_key: true
73       - name: name
74         type: string
75         required: true
76         max_length: 100
77       - name: code
78         type: string
79         required: true
80         unique: true
81         max_length: 20
82       - name: department_id
83         type: integer
84         required: true
85       - name: total_credits_required
86         type: integer
87         required: true
88         default: 120
89     relationships:
90       - type: many_to_one
91         target: Department
92         foreign_key: department_id
93
94   - name: Course
95     attributes:
96       - name: id
97         type: integer
98         primary_key: true
99       - name: title
100        type: string
101        required: true
102        max_length: 200
103       - name: code
104        type: string
105        required: true
106        unique: true
107        max_length: 20
108       - name: description
109        type: string
110        nullable: true
111       - name: capacity
112        type: integer
113        required: true
114        default: 30
115       - name: credits
```

```
116     type: integer
117     required: true
118     default: 3
119   - name: department_id
120     type: integer
121     required: true
122   - name: teacher_id
123     type: integer
124     required: true
125   - name: classroom_id
126     type: integer
127     nullable: true
128   - name: semester
129     type: string
130     required: true
131     max_length: 20
132   relationships:
133     - type: many_to_one
134       target: Department
135       foreign_key: department_id
136     - type: many_to_one
137       target: User
138       foreign_key: teacher_id
139     - type: many_to_one
140       target: Classroom
141       foreign_key: classroom_id
142     - type: many_to_many
143       target: User
144       through: Enrollment
145     - type: many_to_many
146       target: Schedule
147       through: CourseSchedule
148
149   - name: Prerequisite
150     attributes:
151       - name: id
152         type: integer
153         primary_key: true
154       - name: course_id
155         type: integer
156         required: true
157       - name: prerequisite_course_id
158         type: integer
159         required: true
160       - name: is_mandatory
161         type: boolean
162         required: true
163         default: true
164
165   - name: Classroom
166     attributes:
167       - name: id
168         type: integer
169         primary_key: true
170       - name: building
171         type: string
172         required: true
173         max_length: 50
174       - name: room_number
175         type: string
176         required: true
177         max_length: 20
178       - name: capacity
```

```
179         type: integer
180         required: true
181
182     - name: Schedule
183       attributes:
184         - name: id
185           type: integer
186           primary_key: true
187         - name: day_of_week
188           type: integer
189           required: true
190         - name: start_time
191           type: string
192           required: true
193         - name: end_time
194           type: string
195           required: true
196
197     - name: CourseSchedule
198       attributes:
199         - name: id
200           type: integer
201           primary_key: true
202         - name: course_id
203           type: integer
204           required: true
205         - name: schedule_id
206           type: integer
207           required: true
208
209     - name: Enrollment
210       attributes:
211         - name: id
212           type: integer
213           primary_key: true
214         - name: student_id
215           type: integer
216           required: true
217         - name: course_id
218           type: integer
219           required: true
220         - name: enrolled_at
221           type: datetime
222           required: true
223           default: "CURRENT_TIMESTAMP"
224         - name: status
225           type: string
226           required: true
227           default: "active"
228
229     - name: Grade
230       attributes:
231         - name: id
232           type: integer
233           primary_key: true
234         - name: enrollment_id
235           type: integer
236           required: true
237           unique: true
238         - name: score
239           type: float
240           required: true
241         - name: letter_grade
```

```
242     type: string
243     nullable: true
244     max_length: 5
245   - name: graded_at
246     type: datetime
247     nullable: true
248   - name: graded_by
249     type: integer
250     nullable: true
251
252   - name: Assignment
253     attributes:
254       - name: id
255         type: integer
256         primary_key: true
257       - name: course_id
258         type: integer
259         required: true
260       - name: title
261         type: string
262         required: true
263         max_length: 200
264       - name: description
265         type: string
266         nullable: true
267       - name: due_date
268         type: datetime
269         required: true
270       - name: max_score
271         type: integer
272         required: true
273         default: 100
274
275   business_rules:
276     - name: capacity_check
277       description: >
278         Prevent enrollment when course has reached maximum
279         capacity.
280       severity: critical
281
282     - name: duplicate_enrollment
283       description: >
284         Prevent a student from enrolling in the same course
285         twice.
286       severity: critical
287
288     - name: role_based_access
289       description: >
290         Only students can enroll in courses. Only teachers can
291         be assigned as instructors. Only department heads can
292         manage department courses.
293       severity: critical
294
295     - name: prerequisite_check
296       description: >
297         A student must have completed all mandatory prerequisites
298         before enrolling in a course.
299       severity: critical
300
301     - name: schedule_conflict
302       description: >
303         A student cannot enroll in courses that have overlapping
304         schedules.
```

```
305     severity: critical
306
307 - name: credit_limit
308   description: >
309     A student cannot enroll in courses totaling more than
310     30 credits per semester.
311   severity: important
312
313 - name: grade_integrity
314   description: >
315     A grade can only be assigned to an existing enrollment.
316     One enrollment gets at most one grade. Only the course
317     instructor can assign grades.
318   severity: important
319
320 - name: classroom_capacity
321   description: >
322     Course enrollment cannot exceed the assigned classroom
323     capacity.
324   severity: important
325
326 - name: major_requirements
327   description: >
328     Students must complete all courses required by their
329     major program. The system should track progress against
330     major requirements.
331   severity: important
332
333 - name: assignment_deadline
334   description: >
335     Assignments submitted after the due date but within 24
336     hours are marked late (-10%). After 24 hours,
337     submissions are not accepted.
338   severity: nice_to_have
```

Listing 14: Complete large model definition (entities excerpt)

22 Conclusion

22.1 Summary of Findings

This project conducted a systematic comparison between an Intelligent Model-Driven Method and a Pure LLM Method for web application code generation. Using a Student Course Selection Management System as a case study across three scales (small, medium, and large), we evaluated both approaches on structural correctness, compilability, and output consistency.

The key findings are:

1. **Model-driven generation achieves perfect structural fidelity and compilability:** The deterministic skeleton generation guarantees that the output matches the intended architecture exactly, and all generated code passes syntax, import, and build checks (compilability score = 1.000).
2. **Pure LLM generation produces non-compilable code:** Despite generating syntactically correct individual files, all three pure LLM runs produced code that failed to import due to incorrect module organization (compilability score = 0.333).
3. **Model-driven generation is highly consistent:** Cross-run file similarity of 98.4% makes the approach suitable for team collaboration and CI/CD integration.
4. **Pure LLM generation is highly variable:** Cross-run similarity of only 48.1% confirms that LLMs produce substantially different code structures for the same requirements.
5. **The bidirectional feedback mechanism is effective:** All detectable errors were resolved in a single iteration (convergence guarantee with 3-iteration maximum).

22.2 Practical Implications

For practitioners considering LLM-assisted code generation for web applications, our findings suggest:

- **For production systems:** The model-driven approach is recommended when structural consistency, compilability, and reproducibility are important.
- **For prototyping:** The pure LLM approach may be adequate when code quality requirements are lower, but manual correction of import errors should be expected.
- **For team environments:** The high consistency of the model-driven approach facilitates code review, as all team members can generate structurally identical code.
- **For CI/CD integration:** The model-driven approach's deterministic output enables automated pipeline integration without manual verification.

22.3 Final Remarks

This project demonstrates that model-driven engineering principles remain highly relevant in the era of large language models. Rather than replacing formal methods, LLMs are most effective when they operate within well-defined architectural constraints. The combination of deterministic model-to-text transformations for structural code and LLM-based completion for behavioral logic represents a promising direction for AI-assisted software engineering.

The bidirectional feedback mechanism introduced in this work provides a template for how LLMs can be integrated into quality assurance workflows—not as autonomous code generators, but as intelligent assistants that fill specific gaps within a structured development process.